



카테고리 없음

[논문 리뷰] TabNet: Attentive Interpretable Tabular Learning

yuha933 2026. 4. 6. 17:10

1. Introduction

연구 배경

- DNN은 이미지, 텍스트, 오디오 분야에서 두드러진 성공을 보여왔으며, 원시 데이터를 의미 있는 표현으로 효율적으로 인코딩하는 표준 아키텍처들이 빠른 발전을 이끌었다.
- 반면, 테이블 데이터는 가장 흔한 데이터 유형임에도 불구하고, 딥러닝 기반 접근이 상대적으로 덜 탐구되어 왔다. 현재까지도 앙상블 결정 트리의 다양한 변형이 대부분의 응용에서 지배적인 성능을 보이고 있다. 그 이유는 아래와 같다.
 - 표현 효율성 : DT 기반 방법은 테이블 데이터에서 흔히 나타나는 결정 경계를 근사하는 데 효율적이다.
 - 높은 해석 가능성 : 기본적인 트리 구조를 통해 해석이 쉽고, 앙상블 모델에서도 다양한 사후 해석 기법이 존재한다.
 - 빠른 학습 속도 : 트리 기반 모델은 일반적으로 학습 속도가 빠르다.

기존 연구의 한계

- 기존에 제안된 딥러닝 아키텍처들은 테이블 데이터에 적합하지 않은 경우가 많다. 그 이유는 다음과 같으며, CNN이나 MLP 구조의 특징에 기반한다.
 - overparameterization 문제를 가진다.
 - 적절한 inductive bias 부족 문제가 있다.
 - 테이블 데이터의 결정 구조를 잘 학습하지 못하는 경우가 많다.

테이블 데이터에 딥러닝을 적용하려는 이유

- DNN은 트리 기반 방법과 달리 end-to-end 학습이 가능하며, 아래와 같은 장점을 가진다.
 - 이미지 등 다양한 데이터 타입과 함께 효율적인 통합 표현이 학습 가능하다.
 - 수작업 feature engineering 부담이 감소한다.
 - 스트리밍 데이터 학습 가능성이 높아진다.
 - representation learning을 통한 다양한 응용 확장성이 높다.
- 위와 같은 특성들은 도메인 적응, 생성 모델링, 반지도 학습을 가능하게 한다.

본 연구의 제안

- 본 연구는 테이블 데이터를 위한 새로운 표준 DNN 아키텍처인 TabNet을 제안한다. 주요 기여는 아래와 같다.
 - 전처리 없이 raw tabular data 입력이 가능해진다.
 - Sequential Attention 기반 Feature 선택이 가능하다.
 - Local interpretability, Global interpretability의 두 가지 해석 가능성을 제공한다.
 - Self-supervised pretraining을 적용할 수 있다. (이는 tabular 최초 수준이다.)

2. Related Work

Feature selection

- Feature selection : 예측에 유용한 feature들의 부분집합을 선택하는 과정
 - global methods : 전체 학습 데이터에 기반하여 feature 중요도를 계산하는 방식이다. (ex. forward selection, Lasso 정규화 등)
 - instance-wise feature selection : 각 입력 마다 개별적으로 feature를 선택하는 방식이다. (ex. 선택된 feature들과 응답 변수 간의 mutual information을 최대화하는 설명 모델, actor-critic 프레임워크를 활용하여 baseline을 모방하면서 feature selection을 최적화하는 방법 등)

- TabNet : feature selection을 end-to-end 학습에서 직접 제어한다. 하나의 단일 모델이 feature 선택과 출력 매핑을 동시에 수행하며, 이를 통해 더 컴팩트한 표현과 향상된 성능을 달성한다.

Tree-based learning

- DT는 통계적으로 가장 중요한 feature를 효율적으로 선택하는 능력 덕분에 테이블 데이터 학습에서 널리 사용되어 왔다. 또한, 기존 DT의 성능을 개선하기 위해 앙상블 기법이 자주 사용되며, 이는 분산을 줄이기 위한 방법이다. 대표적인 트리 기반 모델들을 아래와 같이 있다.
 - Random Forest : feature의 부분집합을 사용하여 여러 트리를 학습하는 모델이다.
 - XGBoost, LightGBM : 최근 데이터 과학 대회에서 지배적인 성능을 보이는 앙상블 DT 모델이다.

Integration of DNNs into DTs

- DT와 딥러닝을 결합하려는 연구들도 존재했다. 연구들은 아래와 같으며, 이러한 방법들은 자동 feature selection 성능이 저하되는 문제가 있다.
 - Humbrid (Peterson et al. 2018) : DNN 블록으로 트리를 표현하는 방식이다. 다만, 표현 중복이나 비효율과 같은 문제가 발생한다.
 - Soft (neural) decision trees (Yang et al. 2018; Kotschieder et al. 2015) : 미분 불가능한 axis-aligned split 대신 미분 가능한 결정 함수를 사용한다.
- 위의 문제를 보완하기 위한 모델들도 존재했다.
 - Yang et al. (2018) : DT를 DNN에서 시뮬레이션하기 위한 soft binning 함수를 제안했다. 다만, 모든 가능한 분할을 열거해야 하는 비효율이 존재한다.
 - Ke et al. (2019) : feature 조합을 명시적으로 활용하는 DNN 구조를 제안했다. 이는 gradient 기반 학습 대신 knowledge transfer 기반 학습이다.
 - Tanno et al. (2018) : root부터 leaf까지 구조를 점진적으로 성장시키는 방식을 제안했다.
- TabNet : sequential attention을 통해 controllable sparsity를 가진 feature selection을 직접 수행한다는 점에서 기존 연구들과 차별성을 가진다.

Self-supervised learning

- 비지도 학습은 특히 데이터가 적은 환경에서 지도 학습 성능을 향상시킬 수 있으며, 텍스트 분야와 이미지 분야에서 이는 큰 성능 향상을 보여왔다. 이는 적절한 pretraining objective 설계와 attention 기반 딥러닝 구조에 의해 가능했다.

3. TabNet for Tabular Learning

본 연구의 아이디어

- DT가 테이블 데이터에서 좋은 이유 : 테이블 데이터에서는 어떤 feature를 사용하는가가 성능을 좌우하는데, DT는 매 단계마다 가장 중요한 feature를 하나 선택해 해당 feature로 데이터를 나누는 걸 잘한다.
- TabNet의 아이디어 : decision boundary를 feature들의 선형 결합을 만들건데, DNN으로 구현해보자 !!

TabNet의 핵심 특징

- feature selection : 고정된 feature를 사용하지 않고, 각 샘플마다 중요한 feature를 고르는 instance-wise feature selection을 사용한다.
- sequential multi-step architecture : 각 단계가 선택된 feature에 기반해 전체 결정의 일부를 담당하는 순차적 다단계 구조를 갖는다.
- non-linear transformation : 선택된 feature를 비선형적으로 처리하여 학습 능력을 향상시킨다.
- ensemble 효과 : 더 높은 차원과 더 많은 step을 사용하여 앙상블 효과를 모방한다.

TabNet 전체 구조

0. Overall structure

- 입력 feature : 수치형 feature는 raw 값 그대로 사용하고, 범주형 feature는 학습 가능한 임베딩으로 매핑한다. 또한 전체 feature에 대한 전역 정규화는 하지 않고 BN만 적용한다. 모든 decision step에는 동일한 D차원 feature가 입력된다.
- 각 step i 는 이전 step의 출력을 입력으로 받아, 어떤 feature를 사용할지 결정하고 그 결과를 현재 step의 결정 기여와 다음 step으로 전달할 정보로 나눈다.

1. Feature selection

- feature selection 수식 : 이전 step 정보를 보고, 어떤 feature를 쓸지 점수를 매긴 후, sparse하게 선택하는 과정을 갖는다.
 - $a[i-1]$: 이전 step이 만든 정보

- $h_j(a[i-1])$: attention 점수를 만들기 위한 변환 (FC+BN)
- $P[i-1]$: prior 적용으로, 이미 많이 쓴 feature의 점수를 줄이는 역할
- sparsemax : softmax와 달리 일부는 0으로 완전히 제거하여 진짜 선택을 함

$$\mathbf{M}^{[i]} = \text{sparsemax} \left(\mathbf{P}^{[i-1]} \cdot h_i(\mathbf{a}^{[i-1]}) \right)$$

- 실제 적용은 $M[i]$ 와 f 의 곱으로 이루어지며, $M[i]$ 값이 0이면 완전히 제거하여 선택된 feature만 남게 된다.
- prior 수식 : 어떤 feature가 이미 많이 선택됐으면, 다음 step에서 덜 선택되도록 한다.

$$\mathbf{P}^{[i]} = \prod_{j=1}^i (\gamma - \mathbf{M}^{[j]})$$

-
- sparsity loss 수식 : 값이 퍼져 있으면 loss가 크게끔, 몇 개만 선택하면 loss가 작게끔 되어 있어서, 적은 feature만 쓰도록 강제하는 역할을 한다.

$$L_{\text{sparse}} = \sum -M \log M$$

2. Feature processing

- Feature processing 수식 : 선택된 feature를 처리해서 의미 있는 정보로 변환한다.
 - $d[i]$: 현재 step의 판단 결과로, 최종 output에 바로 들어가는 값
 - $a[i]$: 다음 step이 사용할 정보로, 다음 feature selection에 사용됨
 - 내부적으로는 $\text{FC} \rightarrow \text{BN} \rightarrow \text{GLU}$ (중요한 정보만 통과 역할) $\rightarrow$ residual (정보 손실 방지 역할) 의 구조를 가짐

$$[\mathbf{d}^{[i]}, \mathbf{a}^{[i]}] = f_i(\mathbf{M}^{[i]} \cdot \mathbf{f})$$

3. Decision aggregation

- Decision aggregation 수식 : 각 step의 결과를 다 더한다. ReLU를 사용하여 $d[i]$ 값이 음수일 때는 제거, 양수일 때만 기여로 인정한다.

$$\mathbf{d}_{\text{out}} = \sum_{i=1}^{N_{\text{steps}}} \text{ReLU}(\mathbf{d}^{[i]})$$

- 최종 출력 : 마지막 linear layer로 예측값을 생성한다.

$$\text{Output} = \mathbf{W}_{\text{final}} \mathbf{d}_{\text{out}}$$

4. Interpretability

- step 중요도 수식 : step의 output이 크면 중요한 step으로, 작으면 영향이 거의 없도록 설정했다.

$$\eta_b^{[i]} = \sum_c \text{ReLU}(\mathbf{d}_{b,c}^{[i]})$$

- feature 중요도 수식 : 얼마나 많이 선택됐는지, 중요한 step에서 선택됐는지를 모두 반영한다.

$$\mathbf{M}_{\text{agg-}b,j} = \frac{\sum_i \eta_b^{[i]} \mathbf{M}_{b,j}^{[i]}}{\sum_j \sum_i \eta_b^{[i]} \mathbf{M}_{b,j}^{[i]}}$$

5. Tabular self-supervised learning

- 일부 feature를 랜덤하게 가린 후, 나머지 feature로 가려진 값을 복원하도록 학습한다. 이는 BERT의 masked learning과 동일한 아이디어로, 결과적으로 라벨 없이 representation을 할 수 있도록 미리 잘 학습한다.

4. Experiments

학습 설정

- 전처리 : 범주형 feature는 학습 가능한 임베딩을 통해 1차원 스칼라로 매핑하였으며, 수치형 feature는 별도의 전처리 없이 그대로 입력하였다.
- loss : 분류에는 softmax cross entropy를, 회귀에는 mean squared error를 사용하였다.
- optimizer : Adam을 사용하였다.
- initialization : Glorot uniform을 사용하였다.
- 모든 실험에서는 기존 연구들과 동일한 train/validation/test split을 사용하였다.

Instance-wise feature selection

- 6개의 tabular 데이터셋을 사용하였으며, 각 데이터셋은 약 10,000개의 학습 샘플로 구성되어 있다.

- Syn1 ~ Syn3 : 중요한 feature가 모든 샘플에서 동일했으며, 이 경우 global feature selection만으로도 좋은 성능을 보일 수 있다.
- Syn4 ~ Syn6 : 중요한 feature가 샘플마다 다르며, 이때는 global feature selection이 최적은 아니다.

Model	Test AUC					
	Syn1	Syn2	Syn3	Syn4	Syn5	Syn6
No selection	.578 ± .004	.789 ± .003	.854 ± .004	.558 ± .021	.662 ± .013	.692 ± .015
Tree	.574 ± .101	.872 ± .003	.899 ± .001	.684 ± .017	.741 ± .004	.771 ± .031
Lasso-regularized	.498 ± .006	.555 ± .061	.886 ± .003	.512 ± .031	.691 ± .024	.727 ± .025
L2X	.498 ± .005	.823 ± .029	.862 ± .009	.678 ± .024	.709 ± .008	.827 ± .017
INVASE	.690 ± .006	.877 ± .003	.902 ± .003	.787 ± .004	.784 ± .005	.877 ± .003
Global	.686 ± .005	.873 ± .003	.900 ± .003	.774 ± .006	.784 ± .005	.858 ± .004
TabNet	.682 ± .005	.892 ± .004	.897 ± .003	.776 ± .017	.789 ± .009	.878 ± .004

Table 1

- 실험 결과
 - TabNet은 Tree Ensemble, Lasso, L2X보다 성능이 좋고, INVASE와는 비슷한 성능을 보인다.
 - TabNet은 Syn1~3에서는 global 중요 feature를 잘 찾아내고, Syn4~6에서는 instance-wise feature selection으로 인해 성능이 향상됨을 보인다.

Performance on real-world datasets

1. Forest Cover Type (Dua and Graff 2017)

- 목표 : 지도 정보로 forest cover type을 분류하자 !

<i>Model</i>	<i>Test accuracy (%)</i>
XGBoost	89.34
LightGBM	89.28
CatBoost	85.14
AutoML Tables	94.95
<i>TabNet</i>	96.99

Table 2

- 실험 결과
 - TabNet은 기존 트리 기반 모델보다 성능이 우수하다. 특히, 단일 모델임에도 불구하고, 복잡한 AutoML ensemble보다 더 높은 성능을 보인다.

2. Poker Hand (Dua and Graff 2017)

- 목표 : 카드 정보로 포커 핸드를 분류하자 !

<i>Model</i>	<i>Test accuracy (%)</i>
DT	50.0
MLP	50.0
Deep neural DT	65.1
XGBoost	71.1
LightGBM	70.0
CatBoost	66.6
<i>TabNet</i>	99.2
Rule-based	100.0

Table 3

- 실험 결과 : TabNet은 깊은 비선형 처리가 가능하고, instance-wise feature selection 으로 인해 overfitting를 방지한다. 이에 복잡한 규칙 구조를 잘 학습하게 되고, 높은 성능을 보인다.

3. Sarcos (Vijayakumar and Schaal 2000)

- 목표 : 로봇 팔 inverse dynamics 회귀 문제를 풀어보자 !

<i>Model</i>	<i>Test MSE</i>	<i>Model size</i>
Random forest	2.39	16.7K
Stochastic DT	2.11	28K
MLP	2.13	0.14M
Adaptive neural tree	1.23	0.60M
Gradient boosted tree	1.44	0.99M
<i>TabNet-S</i>	1.25	6.3K
<i>TabNet-M</i>	0.28	0.59M
<i>TabNet-L</i>	0.14	1.75M

Table 4

- 실험 결과
 - 작은 모델에서도 성능을 유지하며, 큰 모델에서는 압도적인 성능을 보인다. 이는 모델 크기 대비 효율이 매우 좋다는 것을 알 수 있다.

4. Higgs Boson (Dua and Graff 2017)

- 목표 : Higgs vs. background 분류 문제를 해결하자 !
- 문제 특징 : 데이터가 매우 크다. (10.5M)

<i>Model</i>	<i>Test acc. (%)</i>	<i>Model size</i>
Sparse evolutionary MLP	78.47	81K
Gradient boosted tree-S	74.22	0.12M
Gradient boosted tree-M	75.97	0.69M
MLP	78.44	2.04M
Gradient boosted tree-L	76.98	6.96M
<i>TabNet-S</i>	78.25	81K
<i>TabNet-M</i>	78.84	0.66M

Table 5

- 실험 결과
 - TabNet은 더 적은 파라미터로 비슷하거나 더 좋은 성능을 보인다.
 - structured sparsity를 유지해 효율적인 sparse 구조를 가진다.

5. Rossmann Store Sales (Kaggle 2019b)

- 목표 : 매출을 예측하자 !

<i>Model</i>	<i>Test MSE</i>
MLP	512.62
XGBoost	490.83
LightGBM	504.76
CatBoost	489.75
<i>TabNet</i>	485.12

Table 6

- 실험 결과
 - 시간 feature가 중요하게 선택된다.
 - 휴일 같은 특수 상황에서 instance-wise selection 효과를 보인다.

Interpretability

- Synthetic dataset
 - Syn2에서 output이 X3 ~ X6만 의존하는데 TabNet도 정확히 해당 feature만 선택하는 것을 볼 수 있다.
 - Syn4에서 조건에 따라 다른 feature를 선택해 TabNet이 정확히 instance-wise 선택을 수행하는 것을 볼 수 있다.
- Real-world Dataset
 - Mushroom dataset에서 Odor feature가 가장 중요한 feature인데 기존 방법들은 이에 대한 중요도를 30% 미만으로 잡는 것에 반해, TabNet은 43% 중요도로 본다.
 - Adult dataset에서 기존 연구와 일관된 feature importance를 제공한다.
- 결과적으로, TabNet은 실제로 해석 가능한 feature 중요도를 제공한다.

Self-supervised learning

<i>Training dataset size</i>	<i>Test accuracy (%)</i>	
	<i>Supervised</i>	<i>With pre-training</i>
1k	57.47 $\pm$ 1.78	61.37 $\pm$ 0.88
10k	66.66 $\pm$ 0.88	68.06 $\pm$ 0.39
100k	72.92 $\pm$ 0.21	73.19 $\pm$ 0.15

Table 7

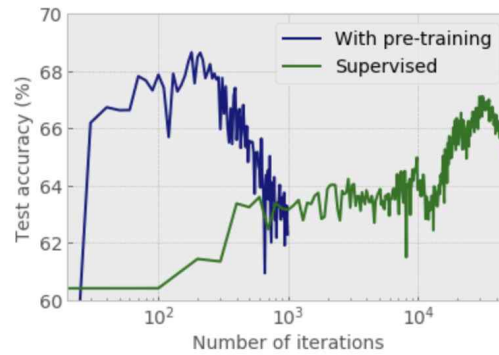


Figure 7

- pretraining이 특히 데이터가 적을 때 큰 효과를 보인다.
- 학습 속도도 더 빠른 것을 볼 수 있다.

5. Conclusion

본 연구의 기여

- 각 decision step마다 의미적으로 중요한 일부 feature를 선택하여 처리하기 위해 순차적 attention 메커니즘을 사용한다.
- instance-wise feature 선택을 통해 모델의 용량이 중요한 feature에 집중적으로 사용되도록 하여 학습 효율을 높인다.
- selection mask를 시각화함으로써 더 높은 해석 가능성을 제공한다.
- unsupervised pre-training이 fast adaptation과 성능 향상에 큰 이점을 제공함을 보인다.